

Possible Exam Questions

Lecture 1: Introduction to Software Engineering

- ***It is said that ‘software doesn’t wear out, but it rusts’. Explain this statement.***
 - **Deteriorates over time** due to constant updates
 - **Ever evolving:** developed what was wanted, delivered it, now clients want more
 - Further from **original implementation** less knowledge devs have
 - **The ppl who write are rarely the ppl who maintain**
- ***“Software is engineered, not manufactured”. Explain this statement.***
 - **Bridge metaphor:** standardized parts, fit different for different situations
 - Every product designed **ground up**, built for different solutions
 - Some parts standardized, but design must be different **to fit differing needs**
 - **Every version of program tailored for specific solutions**
- ***Explain, in detail, the typical ‘wear rate’ (i.e. pattern of error detection) in software that is used for long periods of time.***
 - **All software has bugs:** some bug fixes add bugs
 - also: **different ppl working on it**
 - the longer the software exists the less knowledge ppl have about it
 - **wear rate: change, add, bug fix, change, add, bug fix**
- ***Software Engineering is a ‘layered technology’. Explain this statement.***
 - Underlying everything is **quality** traditional wants underlying layer
 - Even tho not always the main focus
 - Each process has an underlying emphasis on **quality**, then **process**, which has **steps, methods** and each use different **tools**
 - **Companies spend years working on them**
 - **Model** = standardized series of steps
 - ***each layer zooms in on detail** of inner workings
- ***Briefly explain each of the typical framework activities.***
 - Every process will have in some form, sometimes differ in name/crammed together to one
 - Methods + tools vary, **activities stay same** between process models
 - **Communication, planning, modeling, construction, deployment**
 - Communication: find what client needs
 - Rarely they tell you verbatim
 - Planning: plan how to tackle solution, how to solve problem
 - Break down to subtasks, take prob find solution
 - Construction: code generation + testing << testing becoming more important
 - Deployment: training, bug support, call centre, etc
 - Plan for after provided to client
- ***What are three different umbrella activities.***
 - Don’t change step by step (overarching)
 - Project management, technical reviews
 - Formal tech reviews: external review for product (list of req to see)
 - External review to find bugs faster

- Software config management: target of where developed
 - Make sure two ppl aren't solving same prob
- Reusability management
- Measurement
 - How well is model working
 - Measure every time, create new baselines to tweak model
- Risk management
- ***A process framework (or model) is made up of framework activities and umbrella activities. Discuss:***
 - *the difference between a framework activity and an umbrella activity*
 - framework activity is a step within the process
 - umbrella activity is smthn worked on throughout
 - *list the standard framework activities*
 - communication, planning, modeling(analysis of req, design), construction(code gen/test), deployment
 - *what each of the framework activities (generally) involves, and*
 - *some sample umbrella activities.*
 - Umbrella: product/quality/risk/reusability management, measurement, QA, tech reviews

Lecture 2: A Brief Introduction to Process Models

- ***Briefly explain the 4 different process flow models (linear, iterative, evolutionary and parallel).***
 - **Linear (Standard) (waterfall)**
 - Don't go back, each step to completion and not done again
 - **Iterative**
 - Go back or clarification when needed (to a previous step,
 - **Evolutionary (agile) (modern)**
 - Piece by piece to completion, go back and do each step to tiny little portions to delivery
 - **Parallel:**
 - Do a few things at the same time, lots of ppl working at same time
 - When **speed** is key
 - "this is insane. But sometimes it's necessary"
- ***Briefly explain the function of process assessment organizations like ISO.***
 - **ISO** requires a company to adopt a **formal model** and then **fine tune it** to a company
 - **ISO** assesses changes made to model requiring review of **best practices** (company specific)
 - **ISO** mainly focuses on manufacturing **certifies yes quality here**
 - Each org provide techniques for **processes + assess improvements**
- ***Describe, in detail, the waterfall method, including identifying and describing the typical phases.***
 - *Water always goes down*
 - The waterfall method means each of the standard steps are completed **linearly** and **confined** to the step. **You do not go back, you do not go forward**

- Output of one step is input of next, **each step produces deliverables**
- Generate lots of **documentation**, review by external AND client
- **Target build unchanged**: we build this and it is described here

Lecture 3: Requirements Gathering

- ***Describe, in detail, the 6 steps to requirements gathering:***
 - **Inception** : high level understanding + communication of **big picture**
 - Basic understanding of **problem + expectations**
 - Find who wants solution, who will use it, who will see it
 - Remember final users are who determine success
 - Understand problem, desired solution, tech required, issues, costs
 - **Elicitation**: find requirements for **all stakeholders**
 - Suppliers, distributors, government, etc.
 - What input/ **constraints** have from stakeholders
 - **Elaboration**: state scope, create analysis model to ID function
 - Build **initial ideas** and scope and such
 - To do x will cost y and take z things to do, to do A will take C to do and cost B much
 - **Negotiation** : conversation *with* client , client focus
 - Understanding of cost, requirements, deliverables, etc
 - **Specification**: model requirements for development
 - UML
 - **Validation**: Review mechanism
 - client will likely have their own as well
- ***Describe, in detail, what the objectives of the first meeting with the client should be.***
 - **Who** is *really* behind the request, **who** will use what you build, **what** is the \$\$ benefit of a successful solution, is another **source needed**
 - **Stakeholders** (who else to talk to)
 - Recognizing multiple **POV**, different stakeholders have different **priorities**
 - **Keep open mind** early on
 - ID who makes **decisions about (your) money**, success, and where *their* priorities are
 - **Win/ win**
- ***Describe, in detail, the goals of the Elaboration phase of Requirements Gathering.***
 - **Functional description**: describe, in detail, the functionality
 - **Use cases**: desc of actor + system **and their interactions**
 - **Documentation** describe activity and state transitions
 - Describe **behavioral elements** w/ state diagram, activity diagram
 - **As much detail needed to estimate cost**
 - Sometimes high level (agile), sometimes not (what is needed)
- ***What are 5 questions (of the 10 given) that a use case should ask?***
 - **Who** is the primary **actor**, the secondary **actor** (s)?
 - **What** are the actor's **goals**?
 - What **preconditions** should exist before the story begins?
 - What **main tasks** or functions are performed by the actor?
 - **What extensions** might be considered as the story is described?

- What **variations in the actor's interaction** are possible?
- **What system information** will the actor acquire, produce, or change?
- Will the actor have to **inform the system** about **changes** in the external environment?
- What **information** does **the actor desire** from the system?
- Does the actor wish to be **informed about unexpected changes**?

Lecture 4: Project Management Concepts

- ***What are the main objectives of a Statement of Scope?***
 - **Context:** how does software fit into system
 - **Info objectives:** customer visible data objects? Requirements for input?
 - **Function + performance:** transform input to output? Any special performance to address?
 - **id main functionality and data items** clear, concise, unambiguous, understandable
- ***A Statement of Scope becomes the starting point for problem decomposition. Explain this statement and what the end objective is in problem decomposition.***
 - Decomposition of a problem continues until **all functions/classes defined**
 - Consider **characteristics**, determine **degree of rigor**, define **task set** << tasks, products, QA points, milestones
 - End objective is cohesive plan, '**essence**' of project
- ***Briefly define 'degree of rigor' with respect to process management.***
 - How much **detail do we go into**, how much **management applied** to each step
 - **How in depth do we go** in each step of the development process
- ***Describe, in detail, the process of fitting a process model to a particular project.***
 - Select development approach based on projects characteristics
 - Adjust approach to ensure its effective

Lecture 5: Process and Project Metrics

- ***Give 3 reasons why we measure project performance.***
 - **Track status of project:** resources, see how closely original plan is followed
 - ex: two year project, 3 months in you're 10% over ?
 - **Track potential risks:** what problems could occur? Able to find when problem happens
 - **Measure performance :** find bugs/slowdowns before critical
- ***Explain the difference between a metric and a measure. You must define both terms.***
 - **Metric** is a single individual measurement << **targeted measurements**
 - **Measurement** is multiple metrics together
 - We never measure something 1 time, multiple metrics to track 1 thing
- ***What are metrics used for? What should they never be used for?***
 - Used for:
 - Catching errors/problems bc **errors can become exponentially worse**
 - Most metrics are around catching bugs + making sure adhere to plan
 - Regular **feedback for team** to indicate **problems/errors**
 - Improve **process** or **product** (measurements SHOULD find problems)
 - Not for:

- Appraisal of **individuals**, measuring ppl freaks them out
 - Need ppl to *not* lie so their results are accurate
- ***Why do we prefer function-based metrics to size-oriented metrics in estimation?***
 - goal of metric is to **find error** (how many) and **improve model** (best for company)
 - Every new project, choose a couple metrics/tools, edit as needed
- ***Give 3 reasons why we prefer Function Points (FPs) to Lines of Code (LoC) as a metric.***
 - **Function Points** are not **language specific** (LoC more for different ones)
 - Does not penalize shorter implementations, reward useless code
 - **Function Points** independent per solution
 - **Fp** allow easy measure of **reusable components**
- ***Measurement requires resources and is an investment in the quality of the project. Briefly describe how we go about determining what elements of the project require measurement.***
 - Look at your market, decide **what's needed**
 - **Microsoft** example: **quality not important** just get it out (establish self), then focus on quality now

Lecture 6: Estimation for Software Projects

- ***Describe, in detail, each step of the Project Planning Task Set.***
 - **Establish scope:** high level view of what to do
 - **Determine feasibility:** can it be done? (do this early on bc you need to know if project will fail)
 - **Analyze risks:** find what to track (avoid project fail)
 - **Define Required resources:** often scheduling (what skillsets do we need/have , should we hire, what is reusable)
 - **Estimate cost/effort:** decompose problem into subsections and parts
 - if you don't know how to solve a problem it might not be able to be solved
 - **Develop project schedule:** estimate time needed, decompose each part of problem + what needs to be done, find out how long it will take
- ***Give 5 examples of complexity modifiers related to the project as a whole.***
 - How many ppl, how skilled, how good coms, hardware+software how good, how good network, reusable
 - How many ppl
 - How skilled they are
 - How good communication is
 - If hardware/software is good
 - How good is the network
 - How reusable are parts
- ***Explain, in detail, the process of developing an estimate for a project based on function points.***
 - Break down by functionality, **estimate number of points**
 - Organize **average productivity**
 - Determine 'burdened **labor rate**'

- Based on estimated number of **FP** and **historical productivity data**, total estimate
 - Use number of inputs, outputs, inquiries, files, external interfaces
- ***Explain, in detail, how estimation for Agile projects is done.***
 - Agile approach differs because it's **evolutionary**
 - Each **user scenario** (mini use case) considered separately, **decomposed** into set of **software engineering tasks required**, and **each task is estimated separately**
 - Historical data, empirical model, 'experience'
 - Sum estimates for each **task** to estimate **scenario**
 - Develop effort estimate for each **Milestone** (sum of features to create it)
- ***Describe, in detail, the process of the Make-Buy Decision.***
 - **What do I make vs buy?**
 - *make from scratch, buy, reuse, hire contractors
 - Look at with OR without **major changes**
 - Depending on how big/complex, **different est based on different approach**
 - ***flow chart**

Lecture 7: Project Scheduling

- ***Explain the difference between a deliverable and a milestone.***
 - **Milestone:** major functionality, several months per milestone
 - each project has several milestones
 - each milestone has number of **dev cycles** and each cycle has **mini deliverables**
 - **Deliverable:** end of sprint, something to show you're working on (mini checkpoints)
- ***The relationship between effort and delivery time is not linear – a project completed too quickly or too slowly will require significantly more resources than one planned correctly. Explain why this is so, including an explanation of what factors multiply efforts in projects completed too quickly or slowly.***
 - **Too fast = too expensive**, more effort /resources thrown at it to get it done
 - **Too slow = ppl leave**, get sick of it, productivity goes down, turnover goes up (leads to retraining) etc
 - Both lead to increased costs
- ***Writing code is actually a minor part of the creation of a large software project. Explain why this is so and include the breakdown of the typical effort allocation in a project.***
 - Coding + building = 15-20%
 - Front end = 40-50% **communication, design, review, analysis**
 - Testing + installation 30-40%
- ***Define a task network (also known as a dependency chart) and what it is used for. Provide a picture as part of your explanation.***
 - **Each task needs to be billed** each task written in a chart so you know what can be done first/in parallel
 - **The one that looks like a flow chart**
- ***Define a Gantt Chart (also known as a Timeline Chart) and what it is used for. Provide a picture as part of your explanation.***
 - **Timeline chart** each task defined by how long it'll take, organized in order

- Used to define timing and such of when resources are needed
 - ‘can’t start X until Y is done’
- **Briefly describe the process of calculating a project’s ‘burn rate’ and what it is used for.**
 - Burn rate is calculated via resources ‘lost over time’ , compared to resources you have left << money left over example

Lecture 8: Risk Analysis

- **Explain the 4 questions that underly risk assessment.**
 - What can go wrong?
 - What’s the likelihood?
 - What will the damage be?
 - What can we do about it?
- **Explain, in detail, reactive and proactive risk assessment, the differences between them, and which approach we prefer (and why).**
 - **Reactive risk management**
 - Set aside pool of resources for problems **and that’s all they do*
 - Planning for problems takes *resources* and so some companies say nah throw resources at it when it happens
 - Problem w/ this: **panic** << when things **unexpected** ppl freak out
 - **Proactive**
 - **Make a plan**, formal **analysis**, **organization**
 - Make sure **root cause** corrected not risk itself
 - Can track, make plan, understand why
- **Describe, in detail, the 4 steps to risk projection.**
 - Establish scale of perceived likelihood of risk ← **identify risk**
 - What might be significant (ignore any trivial ones)
 - For each risk factor, likelihood of becoming a risk factor **being a prob**
 - Est scale
 - **identify consequences** /, if risk is prb, what is **impact??** < est scale
 - Combine likelihood + risk ,how do we see if it’s a problem and **what can we do to prevent/** if it happens
 - Only spend \$ and time if needs to be dealt with
- **Describe, in detail, the process of building a RMMM plan. In doing so, you must define each of the Ms.**
 - Risk **mitigation monitoring** and **management** plan <<
 - **Sit down.** Com eup with list of risk factors that should be considered
 - Figure out impact of proj, assing value, est probability
 - What **resources** can be used to solve problem
 - Until we run out of money
 - Each **M is a document** < mitigation, monitoring, management
 - Prevention plan, risk detection plan, contingency plan
 - Management plan : if this fucks up wtf do we do ‘**don’t worry we have a plan**’

Lecture 9: Design

- ***What is the purpose of a Use Case Diagram?***
 - **Thread of usage** where we see main functions and tasks by actors
 - Every possible use, not only common uses all of them
 - People designing the solution can use it to make sure everything is addressed
 - Priorities, risks, etc << every use, variation, and actor/agent
- ***What is the purpose of an Activity Diagram?***
 - **Graphical rep of info in a use case**
 - See at a glance what happens in a UC << ppl like pictures, esp upper management
- ***Briefly describe a CRC model – what does it contain, what does it describe and what is it used for?***
 - **CRC Class Responsibility Collaborator**
 - **Collaborator** = what other class that part interacts w/
 - Exists for **maintenance**: when change class A, **what other classes change?**
 - Chain of collaborators: what changes impact what classes
- ***What is the purpose of a Sequence Diagram?***
 - Transition in function **over time** << **top to bottom temporally**
 - Charted w/ ppl, components, etc
 - No tech detail, just graphical
- ***What is the purpose of a State Diagram?***
 - See **what components fit together** where
 - What drives one function to next?
 - w/o tech or detail: just pictures
- ***Why is it important to choose an architecture for a system prior to beginning detailed design (provide 3 reasons)?***
 - Architecture, like software design patterns, allow us to **stay up to date** on what is happening
 - **Maintainable, understandable. Updateable**
 - **Common language** between ppl talking about system
- ***Briefly describe Data Centered Architecture.***
 - **Database**: old, came about after databases became a thing
 - Instead of Many local copies don't work: stale data → **One central data copy** to feed thru
- ***Briefly describe Data Flow Architecture.***
 - **Pipe and filter**, data flows through each component is **COMPLETELY** independent
 - Info comes in, **processed, handed off (independently to next sys)**. ,etc
 - String together series of independent commands/programs
Unix commands
- ***Briefly describe Call and Return Architecture.***
 - Structure system as call to **subroutines** + functions from main program
 - Each sub function calls other programs/functions
 - Fan out like tree, fan into one funct sometimes
 - Procedural programming <this is how structure before OOP
- ***Briefly describe Layered Architecture.***
 - Built from **central core**, add **functionality as layers**
 - Each layer **interacts only w/ below or above**

- OS works like this << highly complex systems that develop over time
- **Discuss the importance of cohesion and coupling in component-level design (you must define each).**
 - **Cohesion:** degree single module works well together, reusability
 - **Coupling:** how closely connected routines are, interdependence

Lecture 10: Architectural Styles

The architectural styles discussed in this lecture are:

Architecture: structure for design

- **Object-Oriented Style**
 - **Components:** Objects (duh) data and associated operations
 - **Connectors:** messages/method invocations
 - **Good:** malleable, system decomposition
 - **Bad:** objects must know each other, side effects from method invocation,
 - Heavy on encapsulation
- **Layered Style**
 - **Components:** each layer acts as a client and server (provider to layers above, consumer of layers below)
 - **Connectors:** protocols of layer interaction
 - **Good:** increased abstraction by layer, evolvability
 - **Bad:** not universally applicable, performance issues, hard to build from scratch
 - OS idea (small core, each light layer introduces new function)
 - Each layer rely directly on layers below
 - Each layer operates independently, only communicate w/ direct above/below
- **Client-Server Style**
 - **Components:** clients and servers, servers do not know ID of clients, but clients know server ID
 - **Connectors:** Remote Procedure Call-based network interaction protocols
 - **Good:** completely independent client/server, no direct link, links established by protocol
 - **Bad:** none on slides
- **Data-Flow Style**
 - **Components:** separate programs executed in order, data items
 - **Connectors:** 'human hand' (ppl walking between programs)
 - Procedural programming, batch sequential
 - Batch sequential was how processing was done up until 70s or so
 - Connectors were humans taking tapes and moving them around
 - Pipe and filter
 - Components are filters, transform input data to output data streams
 - Example: unix
 - **Good:** System behavior, filter addition replacement and reuse, certain analyses
 - **Bad:** batch org, not for interactive applications
 - **Batch-Sequential Style**
 - **Components:** physical data objects, separate programs executed in order
 - **Connectors:** humans taking tapes and moving them

- Pipe and Filter Style
 - **Components:** filters that transform data to streams
 - **Connectors:** pipes , move data streams
 - **Good:** easy filter add, reuse, replace,, concurrent execution
 - **Bad:** interactive applications, organized to batch process info, structural issues (deadlock, parallel processing)
- **Blackboard Style (includes):**
 - **Components:** central data structure (blackboard) components on blackboard
 - **Connectors:** just the blackboard state < hit with commands to do stuff
 - **Good:**
 - **Bad:**
 - - All functionality and all data on one central blackboard
 - Components w/in., data all in, system all in together
 - Hit blackboard w/ commands to do stuff
 - **Rule Based Style :** inference engine parse, determine If it's a fact or query
 - If query, respond with knowledge. If fact, add entry to knowledge
 - **Components:** user interface, inference engine, knowledge base
 - **Connectors:** direct calls or shared memory
 - **Good:** behavior easily modified
 - **Bad:** large num rules involved, understanding interactions is difficult, chain of impact for new rule
 - **Interpreter Style :** parse command, update state maintained
 - Direct control over system state, changes w/ every command
 - **Components:** command interpreter, program state, UI
 - **Connectors:** closely bound w/ calls and shared state
 - **Good:** dynamic behavior, constant architecture, great user end programs
 - **Bad:** highly dynamic, can get complex
- **Mobile Code Style**
 - Package command up to execute it on server, then send it back
 - **Components:** Execution dock (handle receipt of code), compiler/interpreter
 - **Connectors:** Network protocols + elements for packaging code
 - Data elements: rep data as code, program state, data
 - **Good:** great for lightweight processing
 - **Bad:** processing time gotta wait for the result to send back
- **Implicit Invocation Style (includes):**
 - **Components:** methods + events, uses 'event announcement'
 - Listeners register interest
 - **Connectors:** invocation is either explicit or implicit response to event
 - **Good:** component reuse, system can easily evolve at construction and runtime
 - **Bad:** counter-intuitive system structure, relinquish ctrl over system, **no knowledge of response to events, order of response**
 - **Publish-Subscribe Style**
 - Subscribers register/deregister for specific messages/consent, publishers broadcast messages to subscribers
 - **Event-Based Style**

- Independent components emit and receive events over event buses
- Peer-to-Peer Style
 - **Components:** Peers (independent components having own state + control thread), behavior act as either clients or servers
 - **Connectors:** network protocols
 - **Good:** supports decentralized computing, highly robust, scalable
 - **Bad:** protocol issues if custom, redundant connections in topology

Lecture 11: Software Quality Assurance (SQA)

- ***Explain the process of getting ISO certified. Da gold standard***
 - Known **for physical manufacturing**
 - 20 requirements that must be present for an effective QA system,
 - Means they've **followed the processes improvement approach**
- ***Briefly describe the 4 steps of Total Quality Management. (6 marks)***
- ** Japanese style developed by American
 - **Kaizen** << **first step**, most of what we do in class here
 - Continuous **process improvement**, goal is to develop a **repeatable + measurable**
 - Adopt a process model, improve it over time << use template to start
 - Every time we do it we try something different, compare to what last time
 - **atarimae hinsitsu** <<after satisfied w/ kaizen, move to this
 - step back and examine **external factors** that **influence process** (turnover, material quality, etc)
 - identify **variables affecting building process**, what can do to **improve**
 - **kansei**
 - examine **user interaction** with product
 - figure out what customers want + build that
 - UX , how u go in and measure ppls **satisfaction with interaction**
 - **miryokuteki hinshitsu**
 - once company is **satisfied** that they're **building the best product** they can (good with their **techniques**)
 - that's a lot of knowledge that company has built up- on how to build high quality stuff
 - at that pt the companies start **looking to other products**, that they can use the **same techniques to build other (similar) products**

Lecture 12: Review Techniques

- ***Briefly explain the difference between an error and a defect.***
 - **Error :** quality problem found **before release**
 - **Defect:** quality problem found **after release**
- ***Explain the concept of defect amplification.***
 - Software without review yield more error/defects
 - **Illustrate** generation + detection of errors during design and code of software
 - If error persists and is **passed to next step** it gets worse
- ***Explain the concept of pair programming, including what motivates this approach.***
 - **Two heads are better than one** type sh
 - **Put two ppl** in front of **one computer** and have them problem solve **together**

- **Issues:** ppl chit chat, off task, ppl start to think alike
- **Describe the steps of a formal technical review and who participates (i.e. the players).**
 - Formal review bring ppl in from outside to help review
 - **Producer:** the individual developed the product
 - **Review leader:** evaluates product for readiness , generate material to review
 - **Reviews:** spend 1-2h reviewing product
 - **Recorder :** record(in writing) all important issues
 - Review meeting: 3-5 ppl, if agree it's a prob, they send to team that decides 555if it gets fixed
 - Remember reviewing **things**, not **people**

Lecture 13: Testing Conventional Applications

Looking at the specific function of code (review is looking at end result, is code structured / written right) , < testing is looking at nitty gritty

- **What do we look for when designing a “good test”?**
 - High probability of **finding an error**, not **redundant**, best, neither complex or simple
 - Focus **on most important parts** of code
- **Briefly describe white-box versus black-box testing. When doing so, also describe what is each (generally) used for and what (general) techniques are employed by both?**
 - **White box:** know internal workings, ensure ‘all gears’ mesh
 - **All statements/conditions** executed at least once
 - **Black box:** run it through and if this value goes on that value should come out
 - **Does it function?** Does it produce results?
 - **Check boundary conditions, check where might be sensitive to input**
 - **Exhaustive testing:** test *every* data path
 - *Bugs lurk in corners* << only possible way is to test on everything (impossible)
 - *What we do instead is to test boundaries*
- **Explain the process of Basis Path Testing.**
 - Find **Cyclomatic complexity** (measure of how many paths in code)
 - Number of different logic paths to test
 - Use **flow** charts to find simple dec/enclosed areas
 - Derive test cases from this << **apply to critical modules**
- **Explain the concept of equivalence partitioning when choosing data values for a test. Be sure to discuss what values within an equivalence partition should be chosen.**
 - We can't test **every single possible value** so we find the **boundary conditions** that are likely to **cause errors**
 - Standard value inputs for ‘**type**’ of data, find smthn that can ‘rep’ boundary conditions
- **Explain the process of Regression Testing.**
 - **Reexecution** of some subset of **tests every time** to ensure no changes add effects
 - Write a piece of code, it passes all tests, we **check it into dev branch** and such
 - Once code, or anything, has been **tested + reviewed and accepted** it's considered ‘**done**’ << version control

- Keep testing manually OR automated, **re execute of some tests** to ensure no side effects
- ***Explain the process of Smoke Testing.***
 - >>pushing on wells to see if smoke comes out
 - **Rebuild code every day** w/ black box tests automatically running, **make sure still works**
 - **Expose errors** that will keep it from properly building
- ***Explain the process of Comparison Testing.***
 - Separate teams to develop independent versions of an application , all versions tested on same data **keep best results**
 - Extremely expensive, only used when reliability is critical
- ***Explain the process of Top Down Integration, Bottom-Up Integration and Sandwich Testing. Be sure to address the concepts of stubs and drivers.***
 - **Top down:** module tested in **stubs**, pieces at top have code and bottom is all empty
 - **Stubs** replaced depth first, **one at a time**
 - Empty code at bottom, testing code at top, work way down
 - **Bottom up:** detailed parts first, empty '**drivers**' at top,, drivers invoke code at bottom (details)
 - Worker modules grouped into builds/clusters and integrated

Lecture 14: Process Models

Process models, do it in this step and do it correctly and it will be better quality Framework, within is steps and within is activities and within is other sh

- ***Describe, in detail, the Waterfall Model. In doing so, you must describes the phases (steps) and the advantages and disadvantages of the model.***
 - **Communication**, planning, modeling, construction, deployment
 - done in **sequence** , done to **completion**, output of last goes into next
 - **document driven::** big advantage, cuts down crosstalk, **only one that works for big projects**
- ***First briefly describe, and then explain the differences between, evolutionary and throwaway prototyping. Be sure to describe the prototyping process itself.***
 - **Prototyping:**
 - not a process model, it's a **technique**
 - outward **appearances** + behavior, **no function**
 - UI design, mimic functionality for client feedback
 - used often for interface design
 - **Evolutionary prototyping :**
 - System **grows to final functionality**, grows to working system
 - Brings in risk analysis if **spiral**, add new pieces to make function
 - Good for tackling complex, make sure it **could** work
 - **throwaway prototyping :**
 - system is developed, approved, then **final system built from scratch**
 - just for validation of **system requirements**
- ***What are the strengths and weaknesses of prototyping (you must list at least 4 of each).***
 - **Strengths**
 - Give **overview** of project, **show client** what's likely to be involved

- **Weaknesses**
 - Xtra **time** spent on it, xtra **money**
- **Briefly describe the Spiral Model of software development, including the strengths and weaknesses of this approach.**
 - Planning + prototyping + testing to completion for each piece of software, QA as well
 - Each cycle gives more complete version of the software
 - Build, test, deploy for **each piece**
 - **Strengths:**
 - Risk considered at all stages, prototyping at all stages, stepwise still waterfall
 - Constantly tracking risk
 - Build small by small, make some quick prototypes
 - **Weaknesses:**
 - Risk to redo same piece over and over
 - Risk management harder bc changes
- **Describe, in detail, the Unified Process, including the phases (steps) and the advantages and disadvantages of this model.**
 - Similar to **waterfall**, unified, linked to **UML object** model (entities, encapsulations, modularity)
 - Modular **decomposition of system**, hierarchy of abstractions
 - Decompose to deal w/ complexity
 - **jumping back n forth is usually early stages, using nontraditional for risky parts
 - Phases
 - **Inception:** development plan is roughed out
 - High level planning (overlaps with coms + plans)
 - **Elaboration** : refine, fill in details, look at models
 - Use case, analysis, design, implementation, deployment
 - Pound uncertainty out
 - Jumping back and forth between modeling and planning,
 - **Construction:** literally just construction
 - Analysis and design of modules, coded + acquired, tests developed
 - **Transition:** beta test phase
 - support info needed for client to bundle it with it
 - Both construction + deployment (make dev support materials)
 - **Production:** deployment + monitoring

Lecture 15: People Management in Traditional Process Models

- **Explain the MOI approach to team leadership.**
 - **Motivation** encourage tech ppl to produce to their **best ability**
 - **Organization:** mold existing processes to enable concept to product
 - **Ideas/Innovation:** encourage ppl to create/feel creative/give ideas
- **Briefly describe the 4 team organizational paradigm approaches used in team management.**
 - First 3 correspond to amount of input employees have
 - **Closed:** very structured hierarchy of employees

- **Told what to do** and how to do it
 - Management knows **exactly what is going on** at all times, transfer problems
- **Random:** structured on individual initiatives, ‘**self organizing**’
 - Responsibility/authority is **team selected**
 - Problem solving is **a team based** thing, less so management
 - **Hard to trace mistakes** to specific ppl (See above)
- **Open:** structure so controlled but still team innovate
 - **Manager** see and know what’s going on **but team** works together
- **Synchronous:** organizes team members to work on pieces of problem with little communication between them
 - Solving in **parallel** as much as possible
 - Working in **subgroups** that operate independently
- ***What are the three generic traditional team organisation models (hint: the first is democratic decentralized)? Briefly describe each, including their advantages and disadvantages.***
 - **Democratic decentralized** : decisions reached by consensus
 - **Group effort** decisions are made by the team, **communication horizontal**
 - Task coordinator appointed for short durations
 - Most agile uses decentralized → groups self manage and make decisions there
 - **Advantage:** good for morale, innovation
 - **Disadvantage:** takes longer, less quality ctrl, can’t find where things went wrong bc working all together
 - **Democratic Centralized:** defined team lead + secondary leads for subtask
 - Problem solving still group, but implementation assigned by leader
 - **Send off** ppl to solve tasks, can trace errors because of it
 - **Advantage:** good for large projects, quality control w/ communication
 - **Disadvantage:** high management skills required
 - **Controlled centralized:** top-level problem solving, all **by management**
 - Workers told what to do,
 - All communication is vertical, every problem is solved by management, wanna talk to each other? Go to manager first
 - Management aware of everything being done, every decision
 - **Advantage:** Fantastic at quality ctrl, assign responsibility to everyone themselves
 - **Disadvantage:** bad for long term morale, stifles communication/ creativity
 - ‘where ppl with God complexes go’ ‘old school management’
- ***Consistently setting performance expectations too high can lead to a toxic team environment and actually discourage productivity. Explain this statement.***
 - People assume they can get the most out of their team by **constant pushing**
 - **Setting the bar unreasonably high** because it forces them to reach to it
 - However, **repeated exposure to failure** means **loss of confidence and low morale**
 - Set barr too high, they stop trying when they keep missing it

- **Fix:** Goal + stretch goals (give rewards for stretch goals)

Lecture 16: Agile Processes – Underlying Goals, Values and Principles < post internet bubble burst (less expensive, more enjoyable for devs)

- ***Explain the concept of continuous innovation.***
 - Constantly shifting the **product** and **end goal** because we don't know how project will go
 - Deliver on **current customer requirements**, get something out there so the client keeps giving you money
- ***Traditional process models focus on compliance to process while Agile process models focus on feature delivery. Explain this statement.***
 - **Trad:** complete whole product before give out
 - **Agile:** complete small pieces, **add activities** instead of process compliance
 - complete **mvp** so client gets immediate value
 - Focus on small problems, prioritize most important, and streamline process
 - Entire goal of agile is to **get pieces into production**, entire goal of trad is to **complete the process**
- ***One of the primary reasons for generating documentation in traditional process models is proving compliance to process. Explain this statement.***
 - **Traditional** approach focuses on **process** because adhere to process ensures quality
 - Follow the steps and prove you did what you're supposed to do **increases confidence in quality of output**
 - **Prove you did each step correctly**
- ***Iterative development in an Agile environment risks constantly changing requirements. How can one negotiate a budget with the client in such a situation?***
 - Expectation of change requires less money in design+requirements gathering
 - Price for **milestones** not whole system, talk to client about **what piece costs**
 - Price per **unit of development time** (iteration or milestone)
- ***Describe the process of timeboxing.***
 - work in sprints: this development cycle has X many devs who work Y many hours
 - that determines how many **development hours per cycle**
 - grab from **box of features**, fill up schedule until you run out of time
 - variables: length of cycle, hours of development time
 - use **feature list** to fit to plan out hours

Lecture 17: Agile Processes – General Steps

- ***Describe a Product Vision Box.***
 - Vision is high level document to show selling points and features
 - **Vision box** is a design of the packaging, **front + back of box** that you'd sell it in
 - **Purpose of vision** is requirements gathering but not in depth
 - Forces you to sit and **identify 3-4 most important** things
 - Ie- prioritize
 - Cute lil fun thing to show to upper management

- ***Describe in detail an Elevator Test Statement.***
 - Short pitch to show **what's really important** to id target, benefit, advantages
 - Basically, pitch **selling points**
 - Standard format:
 - Id target customer, what needs this product meets
 - Which category it falls into, key benefit/reason to buy
 - Competitive alternative to them, and a statement of '**primary differentiation**'
 - Think bounty
 - People who clean, it helps them clean up spills, it's a paper towel so cleaning supplies, it cleans up spills *the best*, other brands like Kirkland, don't soak up as much spill
- ***What are the main elements of a Product Data Sheet (you must identify at least 6)?***
 - **Identify** of customers/clients, product manager, and project manager
 - Managers on either side you can talk to
 - **Trade-off matrix** : establishes priority of scope, resources, schedule, and defects
 - **Product Objective Statement** short statement of scope, schedule, and resources
 - also might want: exploration factor, delay cost, features, client benefits, performance/quality attributes, architecture, issues/risks
- ***What information is normally included in a Feature Card?***
 - 4x6 **recipe card** to write feature info, can present to user easy
 - What's on:
 - Identifier and name
 - Description in customer terms
 - Feature type (customer/technical)
 - Estimated work effort ← time, requirements, etc
 - Requirements of uncertainty (erratic, fluctuating, routine, stable)
 - Feature dependencies
 - Acceptance tests ← how judge feature is complete, what to make client happy and how can we ensure it happens
- ***Explain the difference between a deliverable and a milestone.***
 - **Deliverable**: things to hand to client
 - **Milestones**: 1-3 months long, major point of functionality
- ***When prioritizing features, what factors should take priority?***
 - Features that are **delivered to the client**
- ***In the Agile approach, it is said that teams are self-managing. Briefly explain what this means.***
 - Guided by the plan but team **decides how tackle/who do what**
 - Adjust plan as necessary, commit to tasks as part of iteration plan
- ***What is the principle of Frequent Integration and what is it trying to achieve?***
 - **Integrate and test as frequently** as possible: daily builds find problems early on
- ***Describe the concept of Ruthless Testing.***
 - Test **during development**, not after
 - **Automate** as much as possible (it'll get done)
 - Integrate testing procedures into build procedures to **catch problems as fast as possible**

- Sooner catch error, cheaper it is to fix
- ***Describe the problem of oscillation when using iterative development approaches.***
 - Make iterations without end because requirements constantly shift
 - **Reworking features constantly**
 - **Prevent:** good vision, continuous feedback

Lecture 18: Agile Process Models

- **Explain, in detail, the Extreme Programming model, including its advantages and disadvantages.**
 - low reliance on methodology/documentation
 - oriented to **small projects**, less than 10 devs, less than a year
 - (oral) **communication + team oriented**, customer and devs on same team
 - Required work products: **code and tests** however, **story cards** are used to guide
 - Index card list feature + details (worked w/ customer)
 - Rely on **testing** and **reject documentation**, stand ups, sustainable pace
 - **Stages:** *exploration* (story cards, feasibility study, estimation), *planning*(stories, release date, iterations to first release), *productionizing*(deployment), *maintenance*(enhance, fix, plan next release)
 - **Advantage:** simple design, pair programming, 'team code own' anyone **can change /improve code**
 - **Disadvantage:** not scalable (only for small projects), uses pair programming
- ***Explain the concept of Pair Programming, including its advantages and disadvantages.***
 - two programmers, one computer , work together while programming
 - pairs might change as tasks change
- ***It is said that Extreme Programming operates under the Principle of Extremes. Explain this statement, providing at least 4 examples of 'taking it to the extreme'.***
 - If something is good, lots is *better*
 - testing good? Write tests for everything
 - If code reviews are good, continuous reviews in pair programming
 - If frequent integration is good, continuous builds on a dedicated machine
 - If short iterations are good, make them 1-3 weeks
 - Customer involvement is good? Customer rep on team
- **Explain, in detail, the Scrum model, including its advantages and disadvantages.**
 - Most popular **development model**, basic premise of agile not as rigid as Xtreme
 - 1-2 week iterations (og 30 days), short time do over and over again
 - **Structure:**
 - Start of every iteration say what we start on w/ **two meetings**
 - Client priorities (goals chosen)
 - How to achieve goals (sprint planning←look at feature list/backlog)
 - At end of iteration: review meeting, demo proj, feedback
 - Teams: team leader **scrum master**, customer rep **product owner**, all others equal (max 7)
 - **Phases:** planning, staging, development, release
 - **Planning:** requ gathering, estimation, little design (overview)

- **Staging:** plan out first iteration, determine overall delivery plan (expect it to change)
 - **Development:** series of iterations << time boxing to find what to do **back and forth with client** to find what needs to get done
 - **Release:** piece by piece w/ training and documentation
 - Continuous in every milestone hand working piece
- **Advantages:**
 - Designed to be flexible, robust base of work products
- **Disadvantages:**
 - Decisions made quickly, requires fantastic management
- ***Scrum is said to be agnostic on documentation and rigor. Explain this statement.***
 - Documentation: **takes no position on it**, doesn't say to or not to use it
 - Use whatever helps for the project you will go into
 - **Rigor** is how detailed you can get (higher degree of novelty higher rigor)
 - Don't follow steps blindly, think **about if will help project**
 - **Scrum leaves this up to the devs, ppl who running project**
- ***Bad decisions are better than no decisions. Explain this statement.***
 - Rather than sit down and talk about it for days, **make a decision quickly work on it for a week** or two and if you made the wrong choice just redo the work
 - If you sit around and talk about it for a week you didn't get anything done
 - One of reasons sprints are so short
- ***Scrum daily stand-up meetings may involve 'pigs' and 'chickens'. Explain this statement.***
 - All of the words used is rugby btw
 - Pigs are **developers** chickens are the **observers** << based on a not very funny joke but it basically says
 - Ppl who have **skin in the game** on a discussion should have more **input** (ppl unaffected don't get a say)
 - Pigs get listened to
- ***What are the 3 questions that are asked at daily stand-up meetings?***
 - What did you do
 - What will you do
 - Any problems you have?
- ***Describe, in some detail, both Extreme Programming (XP) and Scrum. Then highlight (i.e. describe) the main differences between the two approaches to project management.***
 - Scrum is **much more casual**, developer based : if you think it'll help do it, if not don't do it
 - Scrum: Depends a lot on skills of team lead : need to understand tools and what values documents have to process (what to use vs not use)
 - Scrum: small iterations, piece by piece delivery
 - XP all communication, team oriented, pairs keep each other accountable
 - XP : designated tracker for metrics
- ***Describe, in detail, the Agile Modelling (also known as the Agile Unified Process) model, including its advantages and disadvantages.***
 - UML refresh: **50 types of required documentation** (Expensive) but if you do it right your project is good

- Agile Unified : same general approach but for every 50+ UML docs **just decide if it adds value**
 - Here's a toolbox of 50 docs, pick which to use
 - **Advantages:** Scalable, encourages light work but able to use, drive out uncertainty before construction
 - **Disadvantages:** requires good manager, need to know which docs to use and which are useless in situations
- ***What are the guidelines that drive the Agile Modeling model? You must identify at least 4.***
 - Attack risks early and continuously (kill a project blah blah blah)
 - Deliver value to customer early + often
 - Stay focus on dev executable software in early iterations
 - Accommodate change early in project, provoke and change via early dev. Etc

Lecture 19: People Management in Agile Process Models

- ***Explain the difference between leadership and management.***
 - **Leadership** is the pull : how do I ***inspire*** ppl to work to their max
 - **Focus on results** : what did you get done
 - **Management** is the stick approach **depends on control**: pushing ppl, threatening them, tracking what they do and comparing it, etc
 - Sometimes necessary: high complexity = more control needed
 - Poor at handling change, motivating ppl
- ***Adaptive teams are self-organizing. Explain what this involves.***
 - The more ppl feel they have **input** in a project(**ie power+ engagement**) the more they take **ownership** (better work, better production)
 - Manage their own workload + make decision **as a team**, foster individual **accountability**
 - Teams are judged as a team, therefore blame the team
- ***What are the primary responsibilities of a team leader in an adaptive/agile team environment?***
 - **Get the right ppl on board**: They are the ones to organize the right tech / behavioral ppl, **complete control over who's there**
 - **Keep vision in mind**: Responsible for reminding team what place they have in vision and such. **Are we on track with vision?**
 - **Interaction + communication flow** between team: environment were **everyone is equal**, team lead is as equal as possible
 - Ensure **accountability**: make sure if ppl say they'll do something it gets done on time
 - **Steer not control** : don't order people around, group effort so push effort in right direction
- ***The division of responsibility between the customer team and the developer team is an important factor in project decision making. What are the main issues and responsibilities that these teams must deal with and which team has the primary responsibility for each?***
 - **Customer team**: charged with determining features + prioritizing them

- **Provide requirements:** determine **what to build**
 - Customer always right
 - **Product manager:** determine and communicate what to build
 - **Team:** entirely determine features and prioritizing them (define value of each)
- **Development team:** charged w/ delivering results **build product**
 - explain to client how/how not to build but don't argue, customer always right
 - **Project manager:** manage team deliver results
 - **Team:** individuals to deliver results
- ***Explain the concept of servant leadership.***
 - **Manager** should be focused on providing team with **resources needed** to be productive
 - **'role of leaders is to serve team, not order ppl around'** <<get them what they need
- ***Discuss the concept of participatory decision making. Make sure as part of your discussion that you explain how it differs from consensus decision making. In your discussion you must define both participatory and consensus decision making.***
 - How we make decisions in a group where ppl are equal in decision but not in information
 - **Participatory:** everybody participates, gets a chance to state their position
 - **Everyone gets listened to**, talks it out
 - **Manager decide** ultimately (because they get in trouble if wrong, they have more input)
 - **Good for quick+ effective decisions**
 - Different ways : somewhat **democratically**, **by manager** with discussion **or by manager** (don't ask team if you're just gonna veto)
 - Stages:
 - **Framing:** who's involved (should I involve the team or no)
 - **Making:** come up with best solution, pick best ideas from everyone, vote, etc
 - **Retrospective:** at milestones, determine what could have been done better
 - **Consensus:** 'we're gonna talk this out until everybody agrees' ← stupid managers, could talk until the end of time
 - **Also:** if u give ppl right to veto, then take it away bc takes too long, bad
 - **Bad approach** lock into meetings forever

Trad is looking for a formal process of well-defined steps, so you can hand this to anyone and say follow this exactly check that you followed them perfectly and can assume high quality (engineering approach)

Agile don't focus on steps, focus on results: good, high skilled, motivated ppl and set them loose Don't look at how they did it look at what comes out

General Questions to be asked at completion of course:

- Briefly comment on the difference between traditional (i.e. waterfall) and agile processing models. I'm looking for a brief description of each approach and (more importantly) what type of problem each is best suited for. Note that this question says "briefly", so don't spend all of your time on it (and look at how many marks are assigned to this question as compared to others).

Waterfall:

- Water only flows down, don't go back for clarification
- Each step **completed linearly and confined to each step**
- Output of one step is input of next (each step produces deliverables, **documentation**)
- **Target build remains unchanged**
- Focus on following process and generating documentation to prove you did

Agile:

- **Evolutionary style:** piece by piece to completion, go back and do each step again
 - Build working components to completion
 - Do as much requirements gathering as necessary to estimate the cost
 - Estimates per milestone, forces decomposition of project fully
- **Target build evolves** with project, suited for changing requirements
- Focus on **results not the process**, highly team based and highly communication based

- For each of the following scenarios, you are to say what process model (of the ones presented in class) would be most appropriate. More importantly, you are to justify your answer by explaining why this is the proper model, describing the various factors that led to your decision.
 - This question will then contain multiple examples of real-world situations.

Process models available:

- **Linear (Standard) (waterfall)**
 - Don't go back, each step to completion and not done again
- **Iterative**
 - Go back or clarification when needed (to a previous step)
- **Evolutionary (agile) (modern)**
 - Piece by piece to completion, go back and do each step to tiny little portions to delivery
- **Parallel:**
 - Do a few things at the same time, lots of ppl working at same time
 - When **speed** is key
 - "this is insane. But sometimes it's necessary"
-